

# **IA Aplicada com LLMs**

## **Aula 05 — Context engineering dinâmica**

## Onde paramos

A nota 02 da Aula 03 terminou com uma promessa:

*"Naquela nota, o que se monta na chamada. Na aula de agentes, o que se faz com a janela **ao longo da trajetória**."*

E este deck responde à terceira pergunta da aula — por que o laço não está pronto para rodar sozinho?

1. ~~Não sabe parar~~ 2. ~~Não sabe falhar~~ 3. **Não sabe esquecer**

**A cada volta o histórico inteiro é reenviado. Nada nunca sai.**

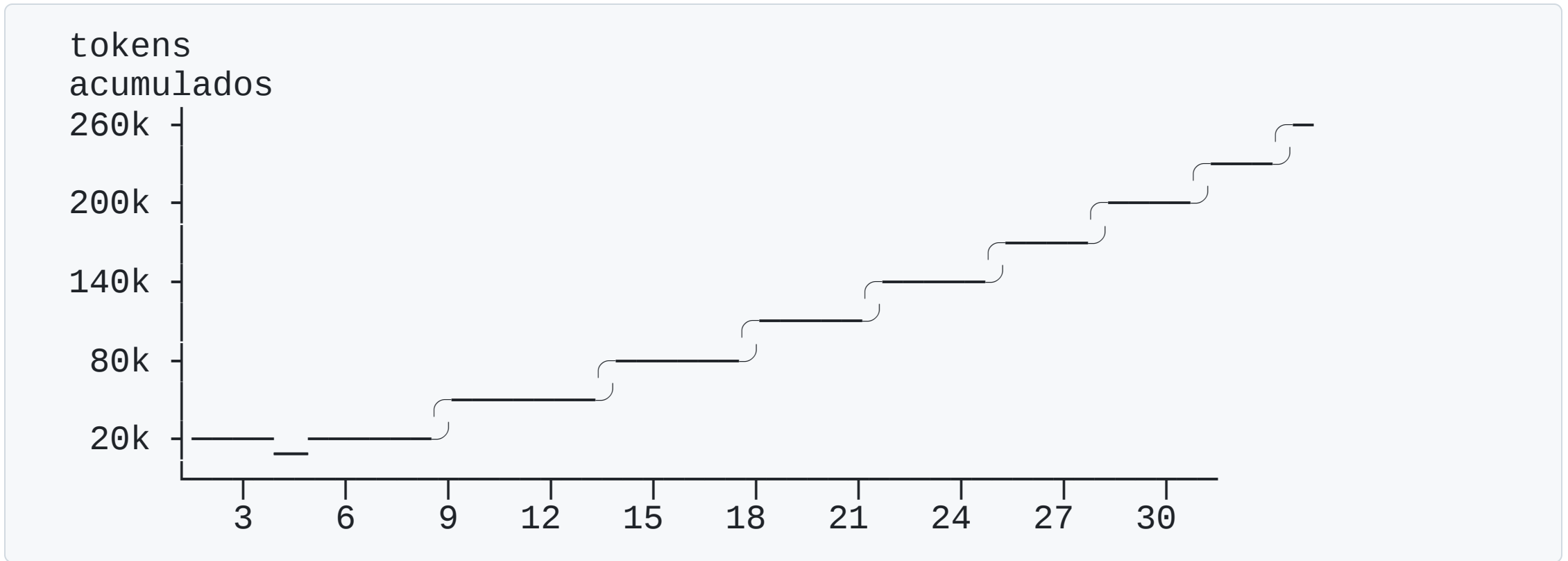
# A aritmética que ninguém faz

Base 1.500 tokens · cada passo acrescenta ~480

| Passo | Enviado neste passo | Acumulado      |
|-------|---------------------|----------------|
| 1     | 1.500               | 1.500          |
| 5     | 3.420               | 12.300         |
| 10    | 5.820               | 36.600         |
| 20    | 10.620              | 121.200        |
| 30    | 15.420              | <b>253.800</b> |

Irrelevante em 3 passos. Dominante em 30.

## A curva



Dobrar os passos não dobra a conta: **quadruplica.**

## **E a leitura menos intuitiva**

**A qualidade cai antes de a janela acabar.**

No passo 25, o objetivo está cercado de 12.000 tokens de observações — exatamente a região do meio, onde o modelo lê pior.

A janela não é um limite que se atinge.

É um recurso que se degrada à medida que é consumido.

# Estático × dinâmico

ESTÁTICO (aula 03)

system  
exemplos  
schema  
documentos  
pergunta



DINÂMICO (hoje)

system  
objetivo  
..... resumo .....  
passos 24-30  
objetivo (again)

preservado  
preservado  
comprimido  
íntegros  
reancorado

decisão única, no código

decisão a cada volta, no programa

Só é possível porque **historico** é **campo do estado**.

## O que enche a janela

| Fonte                           | Volume              | Ainda é útil?           |
|---------------------------------|---------------------|-------------------------|
| <b>Resultados de ferramenta</b> | o maior, de longe   | quase sempre <b>não</b> |
| Raciocínio intermediário        | médio               | raramente               |
| Declarações de ferramentas      | fixo, em toda volta | enquanto ativas         |
| System + objetivo               | pequeno             | <b>sempre</b>           |

O alvo **não é o diálogo**. São os resultados de ferramenta.

# **Parte 1**

## **As quatro táticas**



# 1. Tool clearing — a mais barata

```
{"role": "tool",  
  "content": "[resultado removido] consultar_politica(categoria=refeicao)  
              -> teto R$ 120,00 por refeição; exige nota fiscal"}
```

**Seletiva** (só o que já foi consumido) · **reversível** (o dado completo continua no estado) · **preserva a estrutura** da trajetória · **é código, não modelo** (zero chamadas extras).

**Apagar a chamada inteira faz o agente repeti-la.**

Sem registro da consulta anterior, ele consulta novamente — e a otimização produziu um laço.

## 2. Compaction: reativa × periódica

|                 | Reativa                                    | Periódica             |
|-----------------|--------------------------------------------|-----------------------|
| Dispara         | ao cruzar limiar                           | a cada N passos       |
| Custo           | menor                                      | maior                 |
| Previsibilidade | <b>baixa</b>                               | <b>alta</b>           |
| Risco           | 1 resultado gigante estoura antes do teste | comprimir cedo demais |

Na prática: **periódica como piso, reativa como teto.**

| Passos | Sem compaction | Com (a cada 10) |
|--------|----------------|-----------------|
| 20     | 121.200        | 85.200          |
| 30     | <b>253.800</b> | <b>133.800</b>  |

### 3. Sumarização — o que NUNCA pode sair

| Preservar                        | Por quê                      |
|----------------------------------|------------------------------|
| <b>Objetivo</b>                  | é a âncora — sem ele, deriva |
| <b>Decisões tomadas</b>          | ou o agente reconclui        |
| <b>Escritas executadas + ids</b> | ou registra duas vezes       |
| <b>Erros cometidos</b>           | ou repete a tentativa        |
| <b>Fatos numéricos</b>           | a decisão final vai citá-los |

Pode ir: raciocínio intermediário, resultados consumidos, becos sem saída.

## O prompt de compaction é um prompt de produção

```
PROMPT_COMPACTACAO = """Resuma a trajetória para que outro agente continue do ponto em que ela parou.
```

```
PRESERVE OBRIGATORIAMENTE: decisões e suas razões; ações de ESCRITA executadas com os identificadores; erros cometidos; fatos numéricos.
```

```
DESCARTE: raciocínio intermediário, resultados já usados.
```

```
Não conclua a tarefa. Não invente informação."""
```

Contrato de saída, exemplos negativos, formato — tudo da Aula 03.

**Versionado e testado** como qualquer outro.

**E o preço**

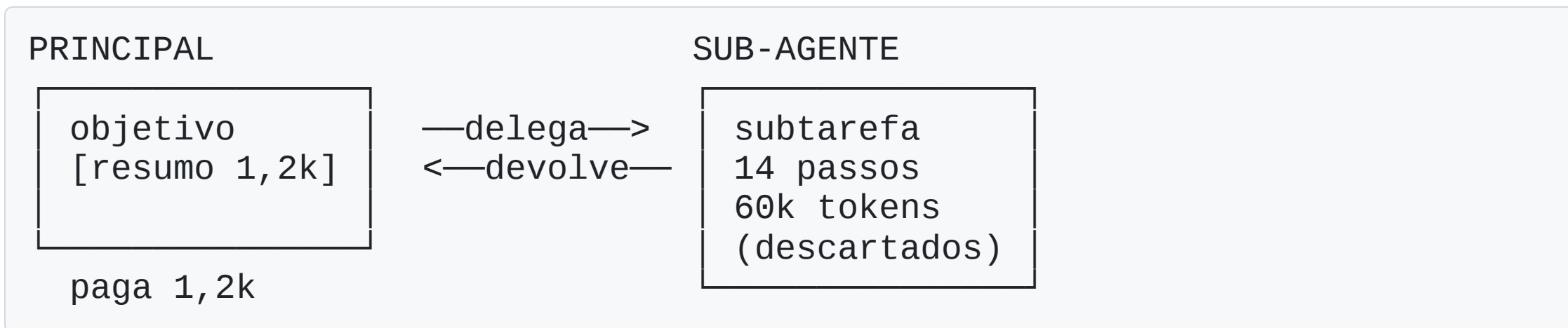
## **Compaction é perda de informação com aparência de continuidade**

O agente segue funcionando, com uma memória que alguém editou.

Quando ele errar por isso, não vai parecer erro de memória.

Vai parecer burrice do modelo.

## 4. Sub-agentes — isolamento



Em vez de comprimir depois, **impede que o contexto se forme.**

**O que o sub-agente não resumiu, o principal nunca saberá.**

Não é compressão — é uma **parede**, e o principal não tem como perceber que faltou algo. Use em subtarefa **fechada e verificável**.

## A ordem de aplicação

| # | Tática                             | Destrói?          |
|---|------------------------------------|-------------------|
| 0 | encurtar o retorno das ferramentas | não               |
| 1 | Tool clearing                      | não — reversível  |
| 2 | Compaction periódica + reativa     | sim, controlado   |
| 3 | Sumarização agressiva              | sim               |
| 4 | Sub-agentes                        | sim, irreversível |

A tática **zero** vale mais que as quatro. Uma ferramenta que devolve 900 tokens quando 60 bastam é defeito de projeto — conserte na origem.

## Uma compaction que quebrou o agente

```
resumo = """O agente analisou a despesa D-4471, consultou a política  
e concluiu que está dentro do teto. O parecer foi elaborado."""
```

Sumiu: **o id do parecer registrado** ( P-1001 ).

Passo seguinte: o agente chamou `registrar_parecer` de novo.

**O que salvou:** a chave de idempotência.

As salvaguardas se cobrem umas às outras. Nenhuma basta sozinha —  
é por isso que a arquitetura tem camadas, e não um remédio.



## Fechando a lista da Aula 03

| Pendência                               | Situação             |
|-----------------------------------------|----------------------|
| <del>padrões de arquitetura</del>       | fechado              |
| <del>estado</del>                       | fechado              |
| <del>múltiplas ferramentas</del>        | fechado              |
| <del>confiabilidade</del>               | fechado              |
| <del>context engineering dinâmica</del> | fechado              |
| <b>memória entre execuções</b>          | aula de memória      |
| <b>MCP</b>                              | aula de MCP          |
| <b>multi-agente</b>                     | aula de multi-agente |

# E uma pendência nova, criada hoje

## evals

O **trace** registrado a partir de agora é a matéria-prima da aula de evals — e da observabilidade, que a aula de produção agrega.

Como sempre: fechar um assunto abre outro.

A diferença é que agora as perguntas são melhores.